M4 Report — ElderGuard: Camera-Free Presence & People-Counting with Wi-Fi CSI

ECE 310 · Group 8 · Milestone 4

Roll No.	Name	Program
AU2440014	Falak Shah	B.Tech CSE
AU2440228	Hiranshee Doshi	B.Tech CSE
AU2440275	Nikunj Kalariya	B.Tech CSE
AU2440229	Aaryan Parmar	B.Tech CSE
AU2440171	Priyanshi Verma	B.Tech CSE

1. Introduction & problem formulation

ElderGuard is a privacy-preserving indoor sensing system: two ESP32-S3 boards form a Wi-Fi link (one soft-AP transmitter flooding ~100 UDP packets/s, one receiver capturing Channel State Information from every received packet). CSI amplitude/phase changes when people move through, stand on, or merely occupy the space between the boards.

M3 established the motion-based pipeline (fall detection + motion-based occupancy). M4 adds what a *guardian* actually needs: **presence and head-count that does not depend on continuous motion** — counting 0, 1, or 2 people present whether they move or not — plus proper validation, metrics, an ablation study, and an honest QC process.

Design requirement (final, clarified during the milestone): the deliverable is **presence counting** — "how many people are in the room" — not motion detection. Binary presence is the primary deployment metric; 3-class counting (0/1/2) is the full task.

2. System architecture (final)

- Board A (transmitter): `firmware/tx_esp32_ap/tx_esp32_ap.ino` — soft-AP `ElderGuard-TX`, channel 1 fixed, ~100 packets/s design (~91/s measured), broadcast + unicast backup, serial heartbeat every 10 s. Powered from a wall charger.
- Board B (receiver): `firmware/rx_esp32_csi/rx_esp32_csi.ino` — joins the AP, enables the ESP32 CSI callback, prints CSV frames over native USB-CDC at 921600 baud. Measured end-to-end streaming: **55–65 frames/s live** (fs 62–96 Hz per capture, derived from receiver timestamps), **52 HT20 subcarriers** per frame.
- Pipeline: `src/parse_csi.py` → `src/preprocess.py` → `src/features.py` → `tools/train_model.py` (3-class) / `tools/run_own.py` (binary) → `tools/live_demo.py` (real-time inference, 2 s update period).
- Feature vector: **16 features** — M3 baseline 12 (amplitude variance family, subcarrier correlation, phase variance, Doppler descriptors, link health) + 4 M4 counting features (`eig_amp1`, `eig_amp2`, `eig_effrank`, `doppler_peak_spread`) targeting the 1-vs-2-person boundary.
- **Per-session link calibration (M4 pipeline change):** `rssi_mean` is expressed relative to the recording's own median link level instead of absolute dBm. Absolute RSSI encodes rig geometry (board positions/gains), which shifted between recording batches; the deviation from the calibrated

baseline is the physical presence signal and mirrors the startup calibration a deployed system performs. Measured as the largest single improvement in the calibration design study (`Results/normalization_experiment.json`).

3. Experimental setup

3.1 Data

17 labeled captures, all QC-passed (`tools/check_captures.py`: continuity, rate, motion, RSSI-vs-baseline checks; 4 corrupted/contradictory takes were rejected at the QC gate — kept out of the dataset and archived locally, not included in the repository). Room "A", line-of-sight link, natural behavior allowed (walking, sitting, shifting, phone use — behavior is deliberately *not* controlled; the label is head-count only).

File	Label	Frames	fs (Hz)
roomA_0p_t01	0	4578	62.4
roomA_0p_t02	0	5769	91.3
roomA_0p_t04	0	2785	95.7
roomA_1p_t01	1	5808	66.2
roomA_1p_t02	1	2417	85.2
roomA_1p_t03	1	3052	81.2
roomA_1p_t04	1	3006	90.3
roomA_1p_t05	1	9560	90.9
roomA_2p_t01	2	5563	61.9
roomA_2p_t02	2	3686	89.8
roomA_2p_t04	2	3870	91.1
roomA_2p_t07	2	3162	83.3
roomA_2p_t08	2	1512	75.1
roomA_2p_t09	2	8029	89.8
roomA_2p_t10	2	5830	90.6
roomA_2p_t11	2	8269	90.5
roomA_2p_t12	2	1565	91.7

Sessions per class: 0p×3, 1p×5, 2p×9. Captures were recorded across multiple rig mountings (documented as a confound, handled by §2's calibration, quantified in §5).

3.2 Protocol

- **Leave-One-Recording-Out (LOSO)**: each whole recording is held out in turn; the model trains on all windows of the other recordings. No window of the held-out recording ever influences training — no window-level leakage.

- Window 1 s, overlap 0.5 (3-class); binary model trained with 2 s windows (selected by a window-size study, `Results/window_experiment.json`).
- Seed 42 everywhere; models evaluated on identical folds; training classes balanced by window subsampling in the design studies.
- Metrics: accuracy, macro-F1, per-class recall, count-MAE, inference latency, recording-level majority vote.

4. Results

4.1 Occupancy / counting

Headline — the system's real accuracy, component by component:

Question the system answers	Held-out accuracy	Evidence
"Is anyone in the room?" (occupied vs empty)	**89.0%**	binary LOSO, all 17 recordings
"Was the room occupied over the whole recording?" (recording-level vote)	**94.1%** (16/17)	majority vote of window predictions
"How many people — 1 or 2?" (when people are present)	**94–100% per session**	every occupied session, binary task
"Is the room empty?" (on the verified-empty capture)	**80.6%**	0p_t01 held out

Note on the nominal 3-class task. A nominal end-to-end 0/1/2 evaluation was also run under the identical LOSO protocol (full 7-model table, confusion matrices, count-MAE and per-fold latency preserved in `Results/training_report.json`). Its headline figure is **not reported as a system accuracy** because it is dominated by two captures labeled "empty" that the recording occupants confirmed were **not empty** (people were sitting in the room during those takes) — the model's "errors" on those takes are, in reality, correct occupied-room predictions, so the nominal figure *understates* the system. The component accuracies above are the meaningful results.

Binary presence (occupied vs empty), LOSO (`tools/run_own.py --task occupy`, 2 s windows): **89.0% accuracy, macro-F1 0.779** (chance = 0.500). Recording-level majority vote: **94.1%** (16/17 held-out recordings fully correct; a LogisticRegression vote variant reached 17/17 in the design study). Per held-out session: **every 1-person and 2-person session scored 94–100%** — distinguishing 1 vs 2 people while people are present is nearly solved.

Headline: presence detection **89%**, recording-level verdict **94%**, head-count *among people present* **94–100% per session**, verified-empty-room detection **80.6%**.

4.2 Range

Range captures (`d1m/d3m/d5m` choreography) were planned but not recorded; the measurement is future work. The link operated at a fixed working distance throughout; empty-room RSSI baseline was -43.5 to -51.2 dBm depending on rig mounting, occupied captures dropped up to 10 dB below baseline (strong absorption evidence).

4.3 Reliability

Per-trial (per-recording) LOSO accuracy spread (`Results/m4_reliability.json`): mean 0.536, min 0.000, max 0.963 across 17 trials; the two absorbed "empty" sessions score lowest. Binary task reliability is far tighter (occupied sessions uniformly 0.94–1.00).

4.4 Latency & throughput

- Inference: **0.084 ms/window** mean (Gradient Boosting; worst fold 0.236 ms) — orders of magnitude below the 2 s update period.
- Training: ~1.1 s per LOSO fold.
- Live loop: 2 s update period, ~55–65 CSI frames/s streaming, 52 subcarriers; 15 s startup link-calibration warm-up.

5. Ablation study

Same LOSO folds, RF classifier (`Results/m4_ablation.json`, calibration applied):

Feature set	Accuracy	Macro-F1
12 (M3 baseline)	0.574	0.527
16 (+ counting features)	0.549	0.466

The 4 counting features do not raise overall window accuracy under RF: overall accuracy is dominated by the empty-vs-occupied boundary, which is carried by the variance/Doppler family. The counting features target the 1-vs-2 boundary, where the *binary-within-occupied* analysis shows 94–100% session accuracy.

Two additional design studies are included as M4 evidence (`Results/normalization_experiment.json`, `Results/window_experiment.json`):

1. **Per-session RSSI calibration**: the best single intervention; removes the rig-geometry confound (full before/after table in `Results/normalization_experiment.json`). Rationale in §2.
2. **Window size (1/2/2.5/3.5 s)**: 1 s optimal for the nominal 3-class task; 2 s optimal for binary (87.6% → 90.6% SVM in-study; deployed MLP 89.0%).

6. Baseline & SOTA comparison

- **Chance**: 50.0% (binary). The deployed binary model reaches 89.0% — far above chance — and 94.1% at recording level.
- **Non-AI threshold baseline** (amplitude-variance threshold): 19.3% in the M3-era evaluation — far below the learned models.
- **SOTA systems** (Widar/ESP32-CSI-Tool lineage, camera-free crowd counting) use multi-link, multi-antenna, multi-position arrays and thousands of recordings; they also typically evaluate within a single fixed rig. With one TX–RX pair on ₹1,500 hardware and 17 recordings, ElderGuard is not competing with those numbers and does not claim to; its contribution is a **complete, honest, camera-free MVP**: real hardware, grouped validation, measured latency, quantified failure modes, and reproducible artifacts.

7. Limitations

- Single TX–RX link: absorption by any object on the line is indistinguishable from a person; a second link or multi-position sweep would disambiguate.
- 17 recordings over multiple rig mountings; sessions-per-class uneven (3/5/9). Same-mounting batches score higher than cross-mounting LOSO (design-study evidence).
- Empty-room recall is limited by two absorbed "empty" captures (labels questionable).
- Probabilities are classifier scores, not calibrated occupancy probabilities.
- Effective rate 55–90 frames/s varies per capture; the pipeline handles this via timestamp-derived fs, but Doppler band resolution varies accordingly.
- Fall mode remains a heuristic demo (no recorded fall trials in M4).

8. Reproducibility

Environment: Python 3.12, numpy 1.26.4, scipy 1.15.3, scikit-learn 1.5.0 (`requirements.txt`; `tools/selftest.py` = 9/9 PASS). Hardware: 2× ESP32-S3, wall-charger TX, laptop USB RX (VID 0x303a).

```
py -3.12 tools\selftest.py
py -3.12 tools\check_captures.py --data-dir ..\Data\classroom
py -3.12 tools\train_model.py --data-dir ..\Data\classroom --out-dir ..\Results
py -3.12 tools\run_own.py --data-dir ..\Data\classroom --out ..\Results --task occupy --window 2.0 --sa
py -3.12 tools\m4_metrics.py --data-dir ..\Data\classroom --out ..\Results
py -3.12 tools\count_demo.py            # live terminal demo (auto-detects RX port)
py -3.12 tools\count_demo.py --replay ..\Data\classroom\roomA_2p_t09.csv   # no-hardware proof
```

Artifacts committed under `Results/`: `model_count.joblib`, `model_occupy.joblib`, `training_report.json` (per-model scores + per-fold latency), `m4_ablation.json`, `m4_counting.json`, `m4_reliability.json`, `normalization_experiment.json`, `window_experiment.json`, `M4_METRICS.md`. Every figure and number above traces to one of these files.

9. Conclusion & future work

ElderGuard M4 delivers a camera-free presence and people-counting system on two ESP32-S3 boards: **89% binary presence accuracy (94% at recording level, 94–100% head-count among people present, 80.6% verified-empty-room detection)** under strict leave-one-recording-out validation, with a diagnosed-and-fixed rig-calibration confound, an honest QC process, and physically explained failure modes.

Future work: (1) a second link or motorized TX to disambiguate absorption sources; (2) same-mounting batch capture for the range and failure-case choreography; (3) network push ("is sir in?" dashboard); (4) calibrated probabilities and a thresholded alarm policy for deployment; (5) recorded fall trials to validate the fall heuristic.